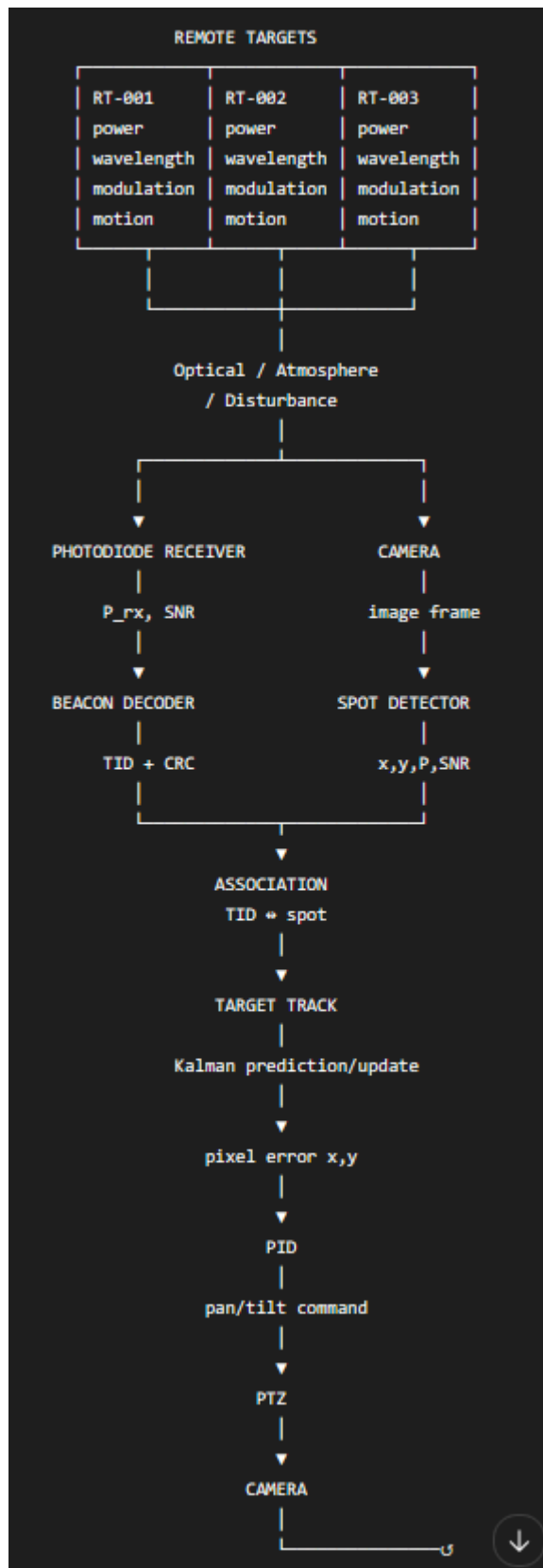




V1 Target Model:

You can have many configured remote targets:

RT-001

RT-002

RT-003

RT-004

Each can have different:

- optical power
- wavelength
- modulation
- beam divergence
- position
- velocity/motion profile
- pointing error
- operational state
- beacon configuration
- atmospheric attenuation

Search

The PTZ initially does a simple coarse scan.

Use the existing fixed grid:

20 positions

4 × 5

640 × 480 FOV

10% overlap

The search controller only needs to answer:

Is there a potentially useful optical target in the current area?

It does not need to solve target selection or tracking.

Overall Architecture:

SEARCH



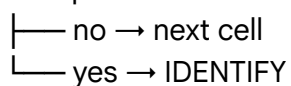
Move camera to grid cell



Observe



Beam present?



Detection & Identification:

Photodiode:

Use received optical power: **Prx**

And preferably: **SNR**

To determine whether a usable optical signal exists.

Example:

Prx = 1.8 mW

SNR = 18 dB

Result = **Optical Candidate Exists**

Then the beacon receiver decodes:

TID = RT-002

CRC = PASS

Wavelength = 1550 nm

Modulation = OOK

Sequence = 184

The identity validator checks:

TID exists in mission registry?

CRC valid?

Not self?

Wavelength acceptable?

Modulation acceptable?

Sequence fresh?

Then you know:

RT-002 is present

Note:

P_{rx} is not the target position. It tells you "A beam is being received"

The beacon tells you "This is RT-002"

The camera tells you "RT-002's spot is here"

Camra Spot Detector:

The camera runs a simple classical detector.

Camera frame → grayscale → background estimate → threshold → connected components → centroid → basic quality checks

Each candidate should produce something like:

```
Detection (  
  X = 421.3,  
  Y = 238.7,  
  Peak = 182,  
  Snr_db = 17.4,  
  Area = 31  
)
```

Association:

For multiple targets.

Suppose the camera sees:

Spot A = (120, 210)
Spot B = (430, 250)
Spot C = (550, 90)

And the beacon decoder has:

RT-001
RT-002
RT-003

V1 needs to associate:

RT-001 ↔ Spot A
RT-002 ↔ Spot B
RT-003 ↔ Spot C

The association should use:

Identity: If possible TID

Spatial Consistency: Compare the spot with target's predicted position

Signal Consistency: Optionally consider: Prx / SNR for additional confidence.

Multiple Targets, One controlled target:

You can observe multiple targets, but the PTZ only actively tracks one.

RT-001 detected
RT-002 detected
RT-003 detected
RT-004 detected



Target Selector



ACTIVE = RT-002

Then only RT-002 feeds the control loop:

RT-002 → Kalman → PID → PTZ

The others can have lightweight last-known information.

Target Selector:

V1 does need selection because there are multiple targets.

But selection should be simple.

Use mission-configurable policy such as:

Priority TID

Strongest P_{rx}

Highest SNR

Nearest predicted target

Example:

RT-001 priority = 1

RT-002 priority = 2

RT-003 priority = 3

Then:

RT-001 available?

YES → select RT-001

You should not have the old elaborate composite score unless you later prove that you actually need it.

Kalman Tracker:

This replaces the α - β filter.

V1 State: $\mathbf{x} = [\mathbf{x}, \mathbf{y}, \mathbf{vx}, \mathbf{vy}]^T$

The tracker performs:

Predict → **Expected x, y** → **Find camera detection** → **Mahalanobis Gate** → **update**
→ **Filtered x, y, vx, vy**

Why this matters?

Suppose RT-002 moves:

320,240

324,241

329,243

334,245

The Kalman filter estimates both: **Position** and **Velocity** and its **Uncertainty**

If the spot disappears temporarily: **Measurement missing** → **Kalman predict** →
Continue toward predicted position

This gives you your coast.

Adaptive Measurement Quality:

High-quality detection: SNR = 25 db → Trust camera measurement more.

Poor detection: SNR = 6 db → Trust prediction more

So the measurement covariance RR can depend on measurement quality.

That is one of the strongest reasons to include P_rx/SNR in the new architecture.

Closed Loop Control:

The control path is:

Camera centroid → **Kalman filtered position** → **Image error** → **PID** → **Pan/Tilt velocity** → **PTZ Dynamics** → **Camera Orientation** → **New Image**

Image error:

$$e_x = x - \frac{W}{2}$$

$$e_y = y - \frac{H}{2}$$

Then:

$$u_{pan} = PID(e_x)$$

$$u_{tilt} = PID(e_y)$$

The PID should use **estimated image position**, not target ground truth.

This is what makes the simulator genuinely closed-loop.

11. COAST

When the active target isn't detected:

```
TRACK
↓
missing detection
↓
COAST
```

During coast:

```
Kalman prediction
↓
predicted target location
↓
PID continues using prediction
```

The covariance grows.

For example:

```
1 missed frame
uncertainty = 3 px

5 missed frames
uncertainty = 9 px

10 missed frames
uncertainty = 22 px
```

The exact values should be calibrated.

12. LOST

V1 should transition to LOST based primarily on:

```
time since valid measurement  
+  
Kalman uncertainty
```

rather than only:

```
N missed frames
```

For example:

```
TRACK  
↓  
COAST  
↓  
uncertainty exceeds threshold  
OR  
timeout exceeded  
↓  
LOST
```

This is much cleaner.

13. REACQUISITION

Once RT-002 is lost:

```
LOST RT-002
  ↓
Kalman predicted position
  ↓
small local search
  ↓
larger local search
  ↓
last-known region
  ↓
coarse raster
```

For example:

```
50 px
  ↓
100 px
  ↓
200 px
  ↓
400 px
  ↓
800 px
  ↓
full scan
```

At each location:

```
P_rx?
  ↓
beacon?
  ↓
TID?
```

And critically:

```
TID == RT-002?
```

If another target appears:

```
TID == RT-003
```

that is **not** a successful reacquisition of RT-002.

14. REACQUIRE → TRACK

Once:

```
P_rx > threshold  
AND  
valid beacon  
AND  
TID == active_target  
AND  
camera spot matches expected region
```

then:

```
REACQUIRED  
↓  
Kalman state reinitialized/updated  
↓  
TRACK
```

You don't need the old elaborate validation state machine.

15. Beacon receiver

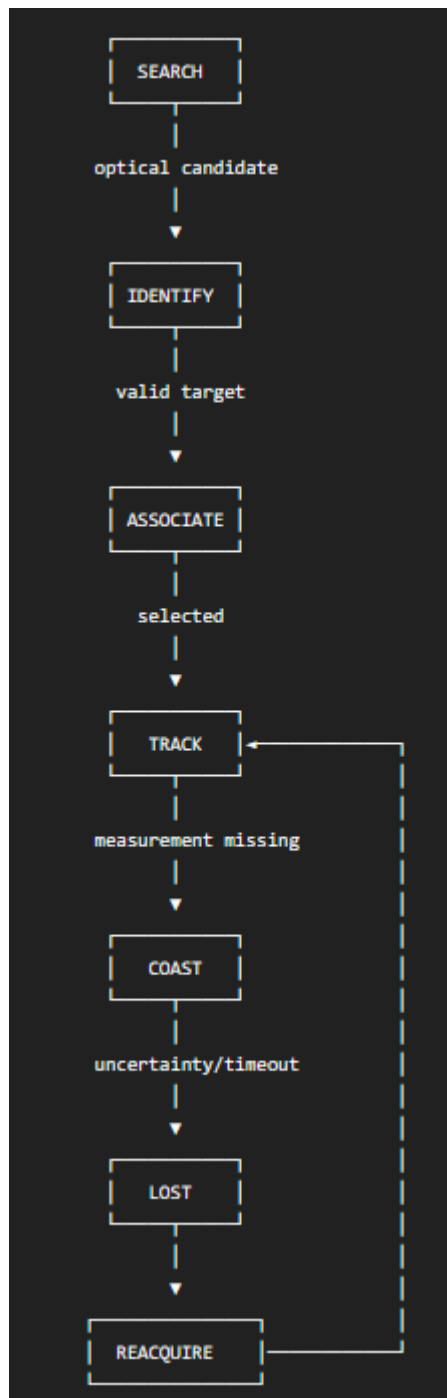
Your existing photodiode receiver is worth keeping.

It provides:

```
P_rx  
SNR  
BER estimate  
beacon synchronization  
TID  
sequence  
wavelength  
modulation  
CRC
```

However, the decoder should remain **parallel to the camera loop**.

The 30 Hz camera should not wait for a complete ~336 ms beacon frame before controlling the PTZ.



FAULT can remain as a safety/system state, but it should not dominate the normal path.

16. What V1 does NOT contain

I would deliberately exclude these from V1:

No AI detector

No YOLO/RT-DETR/CNN.

No quadrant detector

Not necessary for the first working architecture.

No particle filter

Too complicated for the problem.

No EKF/UKF

Your initial state model is linear enough for a standard Kalman filter.

No full multi-object tracking

You support multiple targets, but only one is actively controlled.

No Hungarian assignment

TID + spatial gating is enough.

No elaborate confidence score

Use straightforward gates and signal quality.

No standby pool

You don't need the old standby-pool mechanism in the first implementation.

No complex strike lifecycle

Keep basic invalid-beacon handling.

No MPC/RL

PID is sufficient.

18. V1 module structure

The redesigned `local_terminal/` should roughly become:

```
local_terminal/
├── scan.py
│   │   Coarse PTZ search
├── detector.py
│   │   Camera spot detection
├── comm_receiver.py
│   │   Photodiode + beacon decoding
├── identity.py
│   │   TID/CRC/config validation
├── association.py
│   │   TID ↔ camera spot association
├── tracker.py
│   │   KalmanFilter2D + active TargetTrack
├── selector.py
│   │   Multi-target active-target selection
├── reacquisition.py
│   │   Target-specific local-to-global reacquisition
├── supervisor.py
│   │   V1 state machine
└── models.py
    │   Candidate / TargetTrack / observations
```

The existing physics layers stay outside this:

```
remote_terminal/
camera/
disturbance/
environment/
common/
simulation/
```

19. The V1 data flow

A target can ultimately produce:

`</> Python`

```
TargetObservation(  
    terminal_id="RT-002",  
    fov_x=421.3,  
    fov_y=238.7,  
    p_rx_w=0.0018,  
    snr_db=18.4,  
    timestamp=12.53,  
)
```

Then:

```
Observation  
  ↓  
Association  
  ↓  
Kalman update  
  ↓  
TargetTrack  
  ↓  
PID
```

The active track might contain:

`</> Python`

```
TargetTrack(  
    terminal_id="RT-002",  
    x=420.7,  
    y=239.1,  
    vx=35.2,  
    vy=-4.7,  
    uncertainty_px=3.8,  
    status="TRACKING",  
)
```

That's essentially the core state of your V1 system.

20. What a normal successful run looks like

Example:

```
t=0.0
SEARCH
camera scans grid

t=0.73
P_rx = 1.5 mW
→ optical candidate

t=0.79
Beacon decoded
TID = RT-002
CRC = PASS

t=0.80
Camera finds spot at (470, 260)
→ RT-002 associated

t=0.83
Kalman initialized

t=0.83+
TRACK
Kalman → PID → PTZ → camera

t=4.20
RT-002 temporarily fades
→ COAST

t=4.27
beam returns
→ Kalman UPDATE
→ TRACK

t=8.10
large fade
→ uncertainty exceeds threshold
→ LOST

t=8.13
REACQUIRE near predicted position

t=8.31
P_rx detected
TID = RT-002
→ confirmed

t=8.32
TRACK
```



In one sentence:

**Coarse PTZ search → received-power detection → beacon TID/CRC identification
→ camera spot detection → TID/spatial association → 2D constant-velocity Kalman
tracking → image-based PID → PTZ/camera feedback → uncertainty-based
coast/loss → target-specific local reacquisition → return to tracking.**